>> Intelligent Systems – WC extra <<

Selection of exercises extra preparation for exam

NOTE: this set of exercises is selected based on our assumptions about topics that need more attention after teaching WC1-WC3. There is no time to discuss all topics, but in case you have questions about topics other than in this set of exercises, do not hesitate to ask them.

Try to make all exercise before you come to WC. By this means you will be aware of your own (lack of) knowledge about the topics. Good luck!

Exercises Knowledge Graphs

Exercise 1: Rule-based reasoning: Recall exercise 13 from the previous working group. Prove or disprove whether a knowledge base KB={**C -> -(A v -B), A**} entail **-C** or not, but now using Forward Checking (FC) rule based reasoning. Give the rules that you apply, and write down the closure at the end:

$$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha) \quad \text{commutativity of } \wedge$$
$$(\alpha \vee \beta) \equiv (\beta \vee \alpha) \quad \text{commutativity of } \vee$$
$$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma)) \quad \text{associativity of } \wedge$$
$$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma)) \quad \text{associativity of } \vee$$
$$\neg(\neg\alpha) \equiv \alpha \quad \text{double-negation elimination}$$
$$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha) \quad \text{contraposition}$$
$$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta) \quad \text{implication elimination}$$
$$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) \quad \text{biconditional elimination}$$
$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta) \quad \text{de Morgan}$$
$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta) \quad \text{de Morgan}$$
$$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) \quad \text{distributivity of } \wedge \text{ over } \vee$$
$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) \quad \text{distributivity of } \vee \text{ over } \wedge$$

$$\frac{\alpha \Rightarrow \beta, \alpha}{\beta}$$

$$\frac{\alpha \wedge \beta}{\alpha}$$

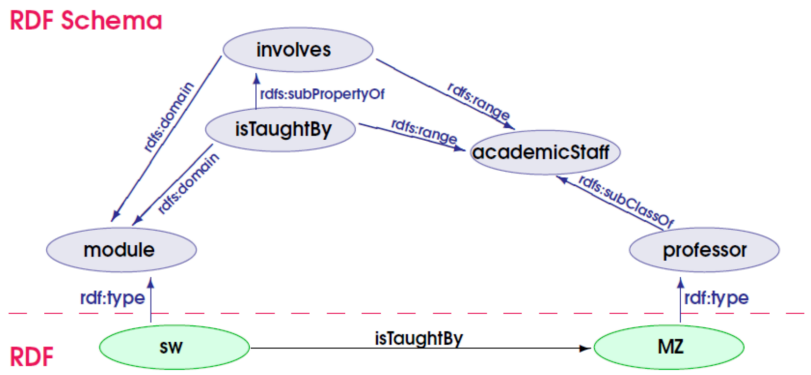
$$\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}$$

First, we apply implication elimination to add -C V -(A v -B), de Morgan to add -C v (-A & B). Then we add (-C v -A) & (-C v B) to the KB. Using and-elimination we can add both (-C v -A) and (-C v B). Now, we can apply -> elimination (or definition) on those formulas to add C->-A, A->-C, C->B and -B->-C. Given A, we can then use Modus Ponens to derive C. There are some more rules you can apply, but the set { C -> -(A v -B), A, -C v -(A v -B), -C v (-A & B), (-C v -A) & (-C v B), (-C v -A), (-C v B), C->-A, A->-C, C->B and -B->-C, -C.} is a subset of the closure, which proves that -C is entailed.

Exercise 2: Rule-based reasoning: Look at the following variant of the previous exercise. Prove or disprove whether a knowledge base KB={**C -> -(A v -B), C}** entail **-A** or not, using Forward Checking (FC) rule based reasoning. Give the rules that you apply, and write down the closure at the end:

Now we use Modus Ponens with C -> -(A v -B) and C, so that we can add -(A v -B) to the KB. Now we use de Morgan to derive -A & -B, and and-elimination to add both -A and -B. This gives the closure KB*={ C -> -(A v -B), C, -(A v -B), -A & B, -A, -B}. As -A ∈KB* we have shown that KB$\vdash_f$-A, which implies KB ⊨ -A.

Exercise 3: Knowledge Graphs: Write down the following Knowledge Graph as a set of triples. Discuss whether there are any non-trivial triples that are entailed.



{SW isTaughtBy MZ, SW rdf:type module, MZ rdf:type professor, …}

…

The one triple that is entailed, which the students should have seen in the lecture, namely MZ rdf:type academicStaff.

There is one more, if you want to you can explain that SW involves MZ, but it has not really been explained in the lecture what rdfs:subProperty means.

Exercise 4: Reasoning with RDFS rules: Suppose the following rule (only) is given.

rdfs9: xxx rdfs:subClassOf yyy AND zzz rdf:type xxx □ zzz rdf:type yyy .

and an RDFS graph:

{ (ex:A rdfs:subClassOf ex:D), (ex:D rdfs:subClassOf ex:C), (ex:B rdfs:subClassOf ex:D), (ex:i1 rdf:type ex:A), (ex:i2 rdf:type ex:B)}

Which triples can be derived?

You can derive: ex:i1 rdf:type ex:D, ex:i2 rdf:type ex:D, we also get ex:i1 rdf:type ex:C by using the rule twice now on ex:i1 rdf:type ex:D.

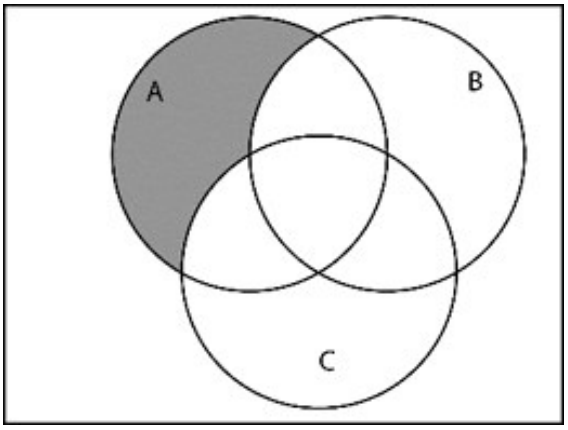
You can derive ex:i2 rdf:type ex:C by applying the rule multiple times: first you derive ex:i2 rdf:type ex:D, and then with ex:i2 rdf:type ex:C.

Exercises Recap

Exercise 1: Sets: Which is an example of disjoint sets?

A. {multiples of two} and {multiples of three}
B. {whole numbers} and {rationalnumbers}
C. {even numbers} and {odd numbers}
D. {multiples of five} and {multiples of ten}

Exercise 2: Sets (Venn Diagrams) What is the definition of the shaded area in the following Venn Diagram.



A & -B & -C

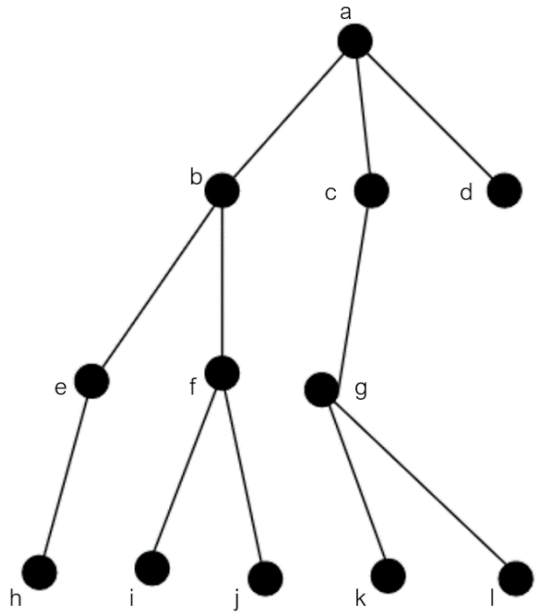
$A \setminus (B \cup C)$ or $A \cap (B \cup C)'$

Exercise 3: Graphs

Draw the graph ({a,b},{(a,a),(b,b)}). How many unconnected subgraphs are there? Write them down in a mathematically correct form. Is this a directed, or undirected graph?

Answer: There are two nodes, a and b, each of them with a directed link to themselves, but no link between the two nodes. There are thus 2 unconnected subgraphs ({a},{(a,a)}) and ({b},{(b,b)}). It is important for the students to understand the notation of a graph as a **pair** (V,E), where V is a **set** of nodes/vertices, and E a **set** of edges (either pairs or sets). In this case, the edges are **pairs** (a,a), so the graph is directed.

Exercise 4 Consider the following tree with 12 nodes.



Answer the following questions:

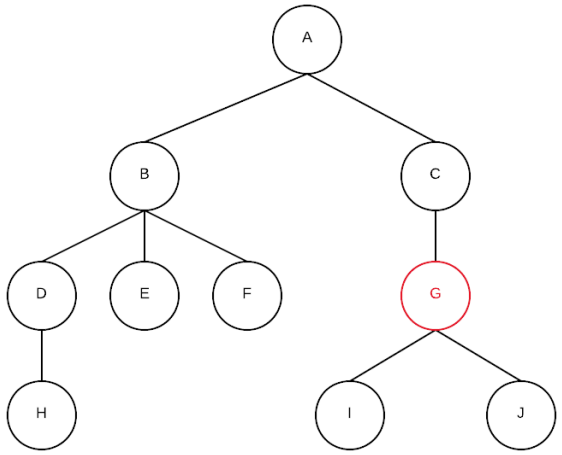
a. Which of the nodes have two children? b,f,g

b. Which of the nodes have no ancestors? a

c. Which are the leaves on level 2 of the tree? None

d. What are the children of the root? b,c,d

Exercise 5:
Let the following search tree, with an initial state A and a state G that satisfies the goal conditions.




```
function TREE-SEARCH(problem,fringe)
return a solution or failure
    fringe
INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
    loop do
        if EMPTY?(fringe) then return failure
        node  REMOVE-FIRST(fringe)
        if GOAL-TEST[problem] applied to STATE[node] succeeds
                then return SOLUTION(node)
        fringe INSERT-ALL(EXPAND(node, problem), fringe)
```

Remember the algorithm for tree-search introduced in the lecture. Please follow the algorithm precisely (as if you were a Computer), given the following instructions. All the notes are added in inverse-alphabetic order in one group to the fringe (i.e. if node A is explored you add (C, B) to the fringe, either at the beginning if it is a FIFO queue, or at the end, if it is a LIFO list.

How many nodes do you need to explore before you find the goal state with DFS, BFD and IDFS? We count a node as explored when it is taken out of the fringe by the REMOVE-FIRST function.

Solution: For DFS you explore A by adding C and B to a new stack (C,B), which means that C is the next node to explore. As the fringe is not empty, and C is not a Goal, C is expanded, and G is added as a single child to the left side of the fringe, which is now (G,B). As this fringe is not empty, the first element G is checked, and is a goal. Only A, C and G have been explored, so the answer is 3.

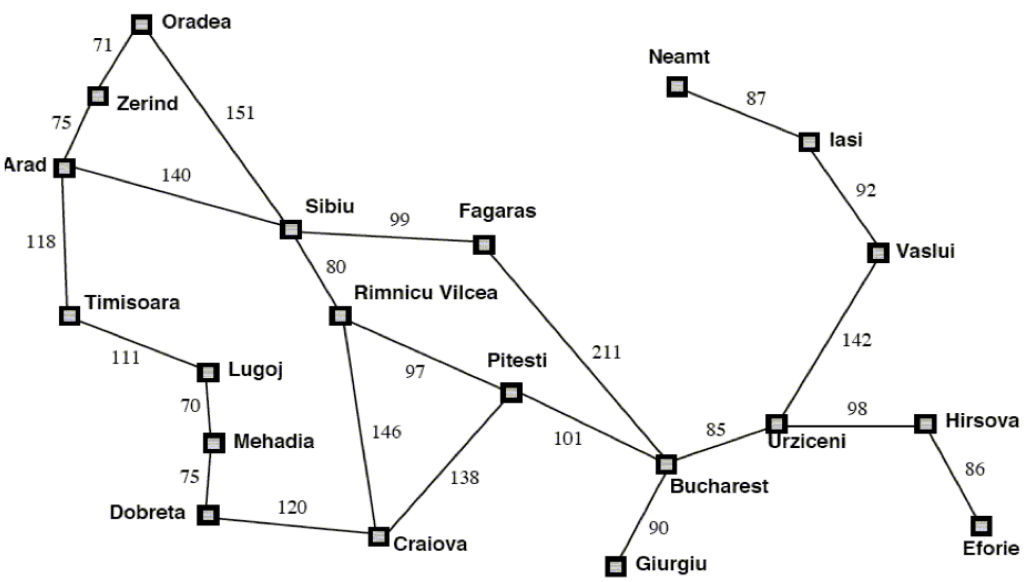
With BFS, you expand A by adding C and B to a queue, which means C is the first node to explore. You then expand C by adding G to the end of the list. Then you explore B, adding the children of B. But you first explore G, which is the solution, so you have to expand 4 nodes before you find the solution. So, this gives the same result.

With DFID, for depth 0, you check (explore) whether A is a solution (1 on depth 0). Then you **explore A** (again) by adding C and B to the fringe, exploring both (3 on depth 1). Then you throw this part of the proof away, and start again until depth 2 with DFS. This means you **explore A** by adding C and B to the stack, which means that C is the next node to **explore**, adding G to the fringe. Then **G is explored** and a solution is found (3 on depth 2). In total 7 nodes have been explored, so many more than with BFS or DFS.

Exercise 6: Informed search

Below you see a map of Romania. On the left connections are depicted with the actual distances. On the right you find information about the straight-line-distance ('hemelsbrede afstand') from that town to Bucharest.

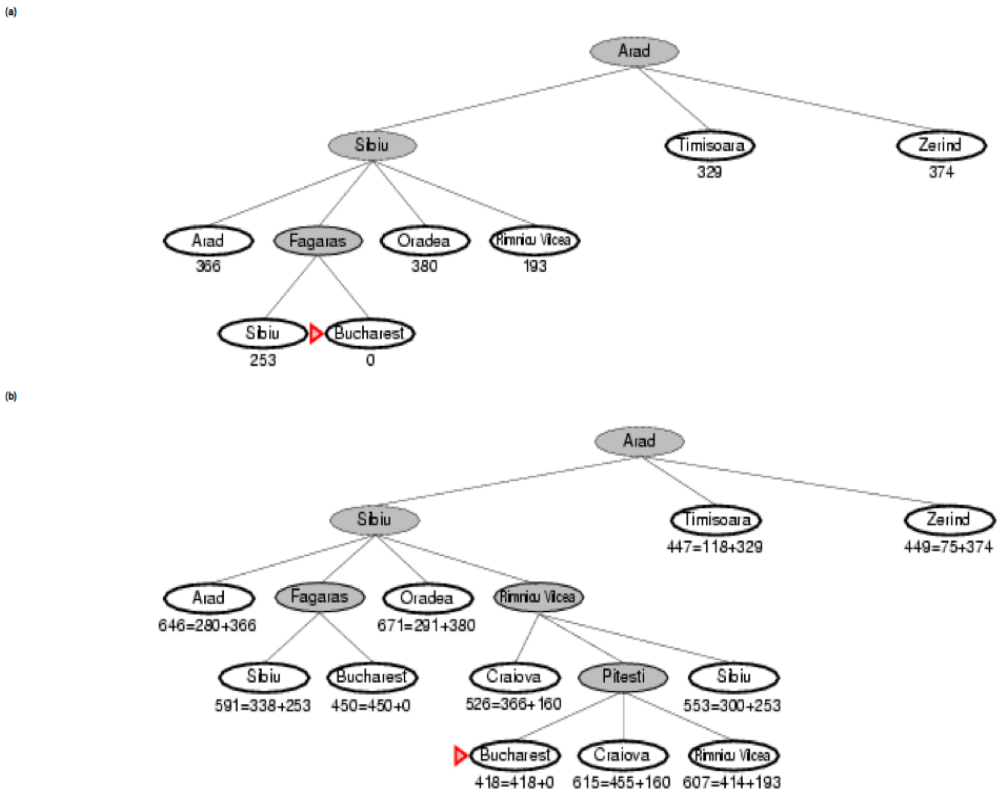
Hieronder vindt je een schematische kaart van Roemenië.



Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnieu Vikea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

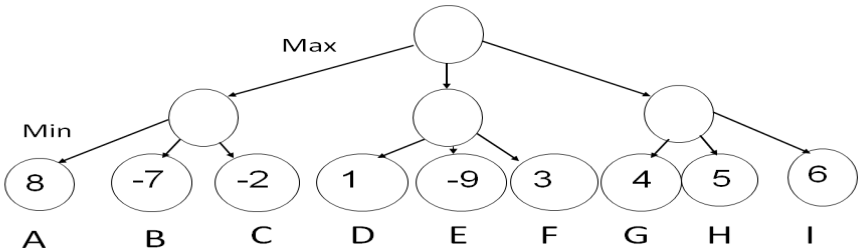
Links staan de verbindingswegen aangegeven, met de werkelijk afstanden tussen de steden. Rechts staat aangegeven de hemelsbrede afstand van elke stad tot Bucharest (de hoofdstad van Roemenië).

a. Show how a **hill-climbing** algorithm would search a route from Arad to Bucharest, where the straight-line-distance is used to estimate the town nearest to Bucharest. Support your answer with a search tree.
   The tree is just: Arad -> Sibiu -> Fagaras -> Bucharest. Only the best node according to the straight-line distance is added to the Fringe, and explored.
b. Do exactly the same as in question a), but now for a A/A* algorithm.
c. Is the applied algorithm in question b) an A or A* algorithm? Explain why.



c. A*, because estimated distance (straight-line distance) is always an underestimation of the real distance (and makes this algorithm therefore admissible).

Exercise 7:
Given is the following search tree:

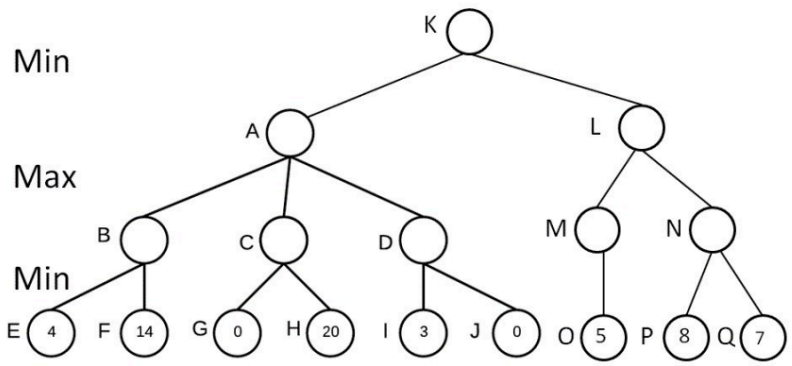


What is the expected value on the root of the tree if Max has the first move? Which nodes can you prune if you use Alpha/Beta pruning?

Expected value is 4. Only F can be pruned.

Exercise 8: Alpha/beta pruning 2

If you apply MiniMax with Alpha-beta pruning, which nodes in the tree can you ignore (prune). More than one answer can be correct.



a. H,J
b. N,P,Q
c. D,I,J
d. H,I
e. M,O
f. L,M,N,O,P,Q

Exercise 9: Truth tables

Complete the following truth table

A	B	C	A v -B	- (A v -B)	C-> - (A v -B)
True	True	True	True	False	False
True	True	False	True	False	True
True	False	True	True	False	False
True	False	False	True	False	True
False	True	True	False	True	True
False	True	False	False	True	True
False	False	True	True	False	False
False	False	False	True	False	True

Exercise 10: Entailment

Check whether the formula A -> (-A v B) entails A & B.

A	B	-AvB	A-> (-A v B)	A&B
True	True	True	True	True
True	False	False	False	False
False	True	True	True	False
False	False	True	True	False

A&B is not entailed, as there are models for A -> (-A v B) in which A&B is not true, namely v(A)=False and v(B)=True, and v(A)=False and v(B)=False.

Exercise 11: Clause Normal Form (CNF)
Translate the formula (A -> (-A v B)) -> -B into Clause Normal Form.
p -> q is rewritten to -p v q
-(A ->(-A v B)) v -B
-(-A v (-A v B)) v -B
(A & -(-A v B)) v -B
(A & A & -B) v -B
(A v -B) & (-B v -B)
(A v -B) & -B

Exercise 12: DPLL for SAT solving
Given a clause set
S = { A v B,
-A v -B,
A v -B v C,
-A v B,
A v -C}
Prove satisfiability, or unsatisfiability, with DPLL. Apply the MOMs heuristics, and assign the value 0 if the "winning literal" occurs with more negation signs than without, and 1 otherwise (equal or less). In case of a draw use alphabetical order, and assign 0 first.

A occurs most often in the four clauses of minimal length A v B, -A v B, -A v B and A v -C. A occurs 2 times with a negation sign, and three times without, so we assign the value 1 to A. This gives the clause set:
{-B}, {B}.
Now B is a unit clause, so that we assign the value 0, which results in an empty clause.
We backtrack and assign 0 to A, which will result in {B}, {-BvC}, {-C}.
There is a draw so alphabetically we choose B, assign it 1 because it is a unit clause, we end up with: {C}, {-C} which is a contradiction. =>. The set is unsatisfiable
We assign 0 to C, leaving us with an empty clause {}, so S is unsatisfiable

Exercise 13: DPLL for Entailment

Prove, or disprove whether **C -> -(A v -B)** and **A** entail **-C** or not, using DPLL with alphabetical clause selection, and negation first.

This is the cooking recipe to be used:
In order to show that **KB ⊨** , we have to show that there is no model for *KB* ∪ {} **(by refutation)**.
We define a procedure **KB ⊢ a** {

- Translate KB into CNF to get **cnf(KB)**
- Translate a into CNF to get **cnf(a)**
- Add cnf(a) to cnf(KB)
- Apply SAT solver (such as DPLL) on **cnf(KB) ∪ cnf(a)** until
  - either model is found (satisfiable) or
  - search exhausted (unsatisfiable)
- If **satisfiable**, return "**NOT ENTAILED**", otherwise "**ENTAILED**".

This is a proof by refutation: you have to transform C -> - (A v -B) and A into Clause normal form, and add the negation of -C. This gives the following sets of clauses in CNF (as C -> - (A v -B) is equivalent to - C v - (A v -B), which is the same as -C v -A & -C v B. So we get
1) -C v -A
2) -C v B
3) A
4) C

DPLL) Now with DPLL we set A to True as a unit clause, which creates a new clause set
1) -C
2) -C v B
3) C

We see directly that C and -C is impossible (if you take the unit clause C, you have to assign C=1, which creates the an empty clause based on {-C}. This means that there is no model, which means that the clause set is unsatisfiable, and -C entailed.